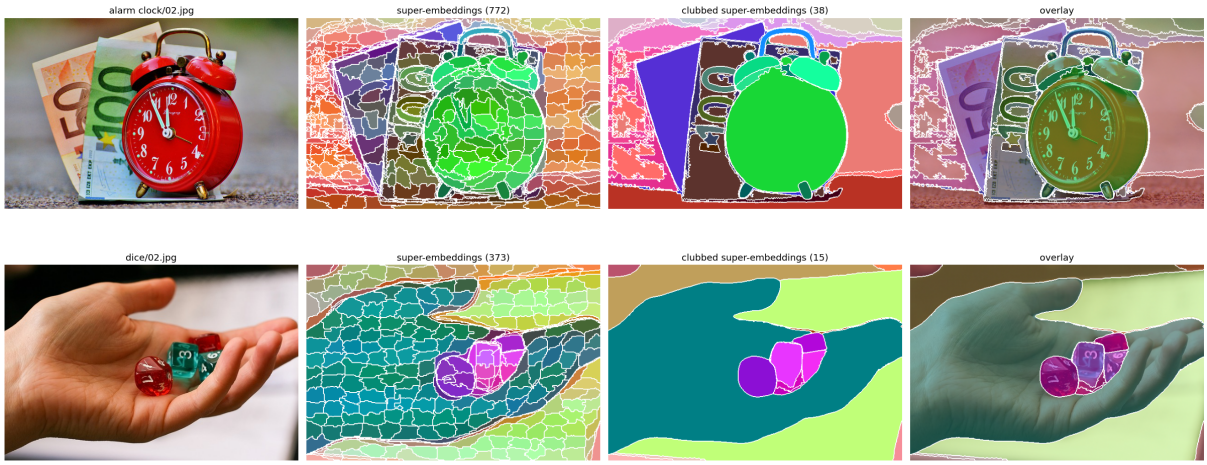
SPAM-Style SAM2 Segmentation Pipeline

Generalizable object clubs & super-embeddings from SAM2

SAM2 embeddings → SNIC superpixels → centroid prompts → subtraction prior → super-embeddings → clubbed super-embeddings



The full hierarchy: original → per-superpixel super-embeddings → clubbed super-embeddings (object level)

This document explains the pipeline end-to-end, every stage of the code, and the significance of each parameter. The design goal was **generalization**: rather than tuning thresholds to one object class, it trusts SAM2's own object proposals and uses geometry (subtraction) plus SNIC superpixels, so it works across object classes without per-class tuning.

1. Why this design

An earlier pipeline grouped SNIC superpixels into objects using ~10 thresholds on object **shape** (solidity, fill) and **embedding similarity**. Those numbers were fit to one class (clocks) and could not generalize — because SAM2 embeddings rate two different clocks (~0.90 similar) as *more* alike than one clock's own face-vs-body (~0.85). So neither shape nor embedding similarity can reliably separate objects.

The fix: let **SAM2 decide what an object is** (it already proposes objects), and turn its overlapping proposals into a clean partition by **subtraction** — no shape/similarity thresholds. SNIC superpixels are kept but now **constrained** by that partition, supplying fine semantic detail inside each object.

The pipeline at a glance

```
image
  | resize so long side <= 1280 (WORK_MAXSIDE)
  v
[1] SAM2 encoder  ->  multi-scale features (encode ONCE)
[2] PCA(20) on features -> SNIC -> semantic superpixels + centroids
[3] prompt SAM2 at each superpixel centroid -> object proposals
[4] NMS -> dedup proposals
[5] SUBTRACTION: paint largest->smallest -> object partition ('clubs')
       + opening separates background from thin boundaries; gaps -> nearest
[6] CONSTRAIN superpixels to clubs (intersection, no parameters)
[7] super-embeddings: 256-d per superpixel
[8] club the super-embeddings -> 256-d per club
  | upsample label maps to original resolution
  v
output: superpixels + super_embeddings + clubs + club_super_embeddings
```

2. Every parameter and its significance

The pipeline has very few *object-dependent* knobs. Most parameters are compute budgets or generic cleanup. The two that matter — both **relative** to image size, so they scale automatically — are `MIN_AREA_FRAC` and `OPEN_RADIUS_FRAC`.

Parameter	Value	Significance
`PROMPT_MODE`	"centroid"	Where to prompt SAM2: at SNIC superpixel centroids (semantic, ~250-600 prompts, faster) vs a blind 3
`PROPOSAL_LEVEL`	"part"	Which of SAM2's 3 masks per prompt to keep: best-IoU (fine, robust) vs largest (whole object, coarser b
`MIN_AREA_FRAC`	0.002	THE main knob. Smallest region kept, as a fraction of image area. Relative, so it scales with resolution.
`OPEN_RADIUS_FRAC`	0.012	Morphological-opening radius (fraction of short side). Separates large background blobs (kept) from thin
`BASE_N_SP / SP_CAP`	250 / 600	SNIC superpixel density (per 640x426) and a hard cap so SAM calls + arrays stay bounded at 4K/8K.
`PCA_DIMS`	20	SAM2 feature channels (352) reduced to 20 for SNIC: decorrelates, denoises, speeds clustering.
`SNIC_COMPACT`	0.005	Near-zero = feature-driven superpixels (follow semantics) rather than a rigid spatial grid.
`SNIC_TARGET`	256	Resolution SNIC runs at on the feature map; labels upsampled afterwards.
`IOU_THRESH`	0.85	NMS: drop proposals overlapping another by >85% (near-duplicates from nearby prompts).
`OVERLAP_TOL`	0.50	Object-mode only: skip a proposal if >50% already covered by a kept one (drops sub-parts). About cove
`OBJECT_MAX_FRAC`	0.90	Object-mode only: ignore a near-total mask (a background click) = reject "the whole scene".
`WORK_MAXSIDE`	1280	Run all logic at <=1280px long side, then upsample. Keeps full-res arrays out of Python loops (the 4K co
`POINTS_PER_BATCH`	64	Decoder batch size for prompts (MPS memory bound).
`MASK_ISLAND_FRAC`	0.0005	Drop disconnected speckle islands inside a proposal.

Stage 0 — Working resolution & single encode

Every image is resized so its long side is ≤ `WORK_MAXSIDE=1280`, then final label maps are upsampled back. This is the key to 4K viability: merge/array operations run on ~1MP arrays instead of 15MP. SAM2 internally resizes to 1024px, so accuracy is unaffected. The encoder runs **once**; SNIC features and centroid prompts reuse the cached features.

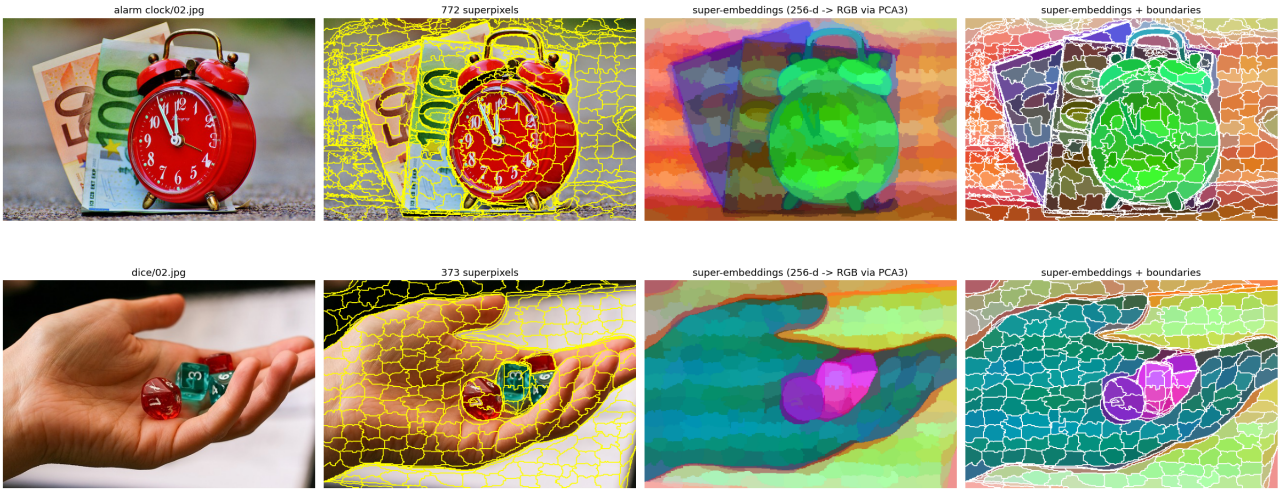
```
def process_image(img_full, predictor):
    H0, W0 = img_full.shape[:2]
    scale = WORK_MAXSIDE / max(H0, W0)
    work = resize(img_full, scale) if scale < 1 else img_full   # downscale big images
    with torch.inference_mode():
        predictor.set_image(work)        # encode ONCE -> predictor._features cached
```

Stage 1-2 — SNIC superpixels on SAM2 embeddings

SAM2's three feature maps (image_embed 256ch + two high_res_feats 64ch & 32ch) are concatenated (352 channels), standardized, and reduced with PCA to `PCA_DIMS=20`. SNIC clusters this 20-D *embedding* map (not RGB), so superpixels follow semantics. `SNIC_COMPACT=0.005` makes clustering feature-driven. The high-res feats give crisp boundaries (image_embed alone is only 64x64).

```
def pca_feature_map(predictor, H, W):
    feats = predictor._features
    combined = concat([ to_t(feats['image_embed']),        # (256,256,256) semantic
                        to_t(feats['high_res_feats'][1]),   # (256,256,64)  mid detail
                        to_t(feats['high_res_feats'][0]) ]) # (256,256,32)  fine edges
    flat = StandardScaler().fit_transform(combined.reshape(-1, 352))
    pca  = PCA(20).fit_transform(flat)                      # decorrelate + denoise
    return normalise(pca).reshape(256, 256, 20)

def snic_superpixels(predictor, H, W):
    feat = pca_feature_map(predictor, H, W)
    labels,_,_ = snic(feat.tolist(), n_superpixels(H,W), compactness=SNIC_COMPACT)
    return upsample(labels, H, W)        # semantic superpixel label map
```



SNIC superpixels (col 2) and their super-embeddings projected to RGB (col 3): semantically similar superpixels get similar colours — clock body, bills and background are each coherent.

Stage 3 — Centroid prompts -> SAM2 object proposals

Each superpixel's centroid becomes a point prompt — one semantic prompt per region, far fewer than a blind 1024-point grid. SAM2 returns 3 masks per prompt; `_choose_granularity` picks one: best-IoU for *part* level (robust) or the largest for *object* level. Each mask is cleaned (fill holes, drop islands).

```
def sam_point_proposals(predictor, pts, img_area):
    for chunk in batches(pts, POINTS_PER_BATCH):     # decode in chunks (MPS memory)
        masks, ious, _ = predictor._predict(coords, labels, multimask_output=True)  # (B,3,H,W)
        for b in range(B):
            k = _choose_granularity(masks[b], ious[b], img_area)  # part=best-IoU / object=largest
            proposals.append( clean_mask(masks[b,k]), ious[b,k] )
    return proposals
```

Stage 4 — NMS

Nearby centroids on one object give near-identical proposals. NMS keeps the higher-score one and drops any overlapping it by > `IOU_THRESH=0.85`.

Stage 5 — Subtraction into a partition (the core idea)

SAM2 proposals overlap and don't tile the image. To make a partition, paint them **largest** → **smallest** so the smallest (most specific) object **wins** each overlapping pixel — equivalent to subtracting smaller objects from larger ones. This is why the method generalizes: two touching dice stay separate because SAM2 already proposed them separately; there is no 'are these the same object?' threshold to tune.

```
def aggregate(proposals, H, W):
    objs = [m for m,s in proposals if m.sum() >= MIN_AREA_FRAC*H*W]   # min-area filter
    objs.sort(key=lambda m: -m.sum())            # largest first ...
    label = full(-1)
    for i,m in enumerate(objs): label[m] = i     # ... smaller painted last -> wins overlaps

    unlabeled = (label == -1)                    # pixels no proposal covered
    opened = binary_opening(unlabeled, disk(OPEN_RADIUS_FRAC*min(H,W)))
    for blob in components(opened):              # large blobs = background/surfaces
        if blob.sum() >= min_area: label[blob] = new_id
    holes = (label == -1)                        # remaining thin boundaries
    label[holes] = nearest_label(holes)          # assign to nearest region (distance transform)
    return merge_slivers(label, min_area)        # absorb any leftover fragment
```

Morphological opening handles the leftover pixels: opening removes thin structures, so what survives is large *background* (kept as regions) and what's removed is thin *inter-object boundaries* (assigned to the nearest region).

Stage 6 — Constrain superpixels to the prior

A pure intersection with **no parameters**: every superpixel is split at object boundaries so it lies inside exactly one club. The prior supplies object boundaries; SNIC supplies fine detail inside them; no superpixel bleeds across an object edge.

```
def constrain_to_prior(segs, prior):
    combo = segs * (prior.max()+1) + prior       # unique (superpixel, club) pairs
    fine, sp_to_club = relabel(combo), {}
    for unit in fine: sp_to_club[unit] = the club it lies in
    return fine, sp_to_club
```

Stage 7-8 — Super-embeddings and clubbed super-embeddings

Super-embedding = the embedding analogue of a superpixel: each superpixel gets one 256-d SAM2 `image_embed` vector (mean over its pixels, L2-normalised). The image is now N units, each a (mask, 256-d vector) pair.

```
def super_embeddings(predictor, segs):
    embed_up = upsample(predictor._features['image_embed'])   # (H, W, 256)
    for sid in unique(segs):
        v = embed_up[segs==sid].mean(0); mat[i] = v / ||v||   # 256-d, L2-normalised
    return ids, mat, sizes
```

Clubbed super-embeddings build object-level vectors bottom-up: each club's vector is the pixel-size-weighted mean of its member superpixels' embeddings. This equals the pixel-based club embedding (verified at cosine 0.9999 across 930 clubs) but is computed from the super-embeddings.

```
def club_from_super(sp_ids, sp_mat, sp_sizes, sp_to_club):
    for club in clubs:
        rows = members of this club
        v = weighted_mean(sp_mat[rows], sp_sizes[rows]); out[club] = v / ||v||
    return out      # 256-d per club
```

The three-level pyramid (final output)

```
res = {
  'superpixels':           (H,W) int32        # spatial map
  'superpixel_ids':        (N,)
  'super_embeddings':      (N, 256)           # 256-d per superpixel   [pixel -> super]
  'label_map':             (H,W) int32        # object clubs
  'club_super_embeddings': {club: (256,)}     # clubbed from super-emb  [super -> club]
  'club_embeddings':       {club: (256,)}     # pixel-based (cos ~ 1.0 with above)
}
```

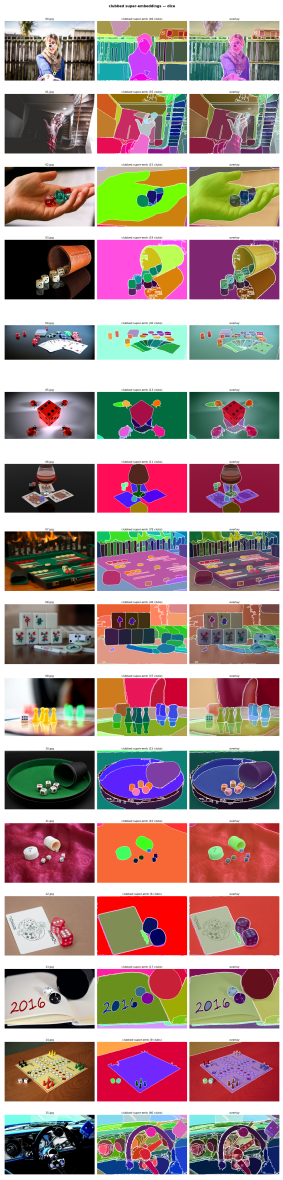
3. Generalization results

The same configuration (no per-class tuning) was run on five very different image types — the central claim: object grouping comes from SAM2 + geometry, not class-specific thresholds.

Parameter	Value	Significance
`Dataset`	imgs	Result
`Alarm clock`	16	clubs 18-56; clocks, bills, backgrounds cleanly separated
`Dice`	16	clubs 6-80; dice separated from hands/tables for free
`Clover`	10	clubs 12-56; clovers vs hands vs grass/fabric
`Soap bubble`	9	clubs 10-53; bubbles caught where they have a visible rim
`Plane (4K)`	1	22 clubs at 4767x3249 in ~73s; fuselage/tail/sky distinct

Embedding correctness verified numerically: clubbed super-embeddings match the pixel-based club embeddings at cosine **0.9999** (min 0.9975) across 930 clubs — confirming the bottom-up aggregation is sound.

Example — dice (clubbed super-embeddings)



Dice dataset: each object coloured by its 256-d clubbed super-embedding. Dice consistently pop out from hands and tables.

4. Honest limitations

- **Object-vs-scene granularity is ambiguous without supervision.** 'part' mode gives clean fine regions; 'object' mode is coarser but can over-merge a single dominant subject (a lone plane filling the frame collapses to one region). A per-use-case choice, not a bug.
- **Low-contrast/transparent objects** (faint soap bubbles) can be absorbed into the background — a limit of any segmentation on near-invisible objects.
- **Granularity tracks scene complexity**: busy scenes yield more clubs. For a fixed object count, cluster the super-embeddings downstream.

5. Usage

```
# run on a folder
python pipeline_spam.py --image_dir '/path/to/images' --output_dir '/path/to/out'

# load results
import numpy as np
d = np.load('out/<cat>_full.npy', allow_pickle=True).item()
r = d['02.jpg']
Sm = r['super_embeddings']         # (N, 256)  per-superpixel
C  = r['club_super_embeddings']    # {club: (256,)}  per-object

# cosine similarity is a dot product (all vectors L2-normalised)
sim = float(C[1] @ C[2])
# nearest club across images:  gallery @ query  -> argmax
```

Summary. SAM2 embeddings → SNIC superpixels → centroid prompts → subtraction prior → super-embeddings → clubbed super-embeddings. It replaces ~10 brittle, class-specific thresholds with SAM2's own object proposals plus two relative parameters, and delivers a three-level representation (pixel → superpixel → club), each carrying L2-normalised 256-d SAM2 embeddings ready for retrieval or clustering.